

Unsupervised Learning

Discover Hidden Patterns Without Labels

■ CLUSTERING & DIMENSIONALITY REDUCTION

■ LEARNING OUTCOME

By the end of this module you will apply K-Means and DBSCAN clustering, reduce dimensions with PCA and t-SNE, detect anomalies, and interpret unsupervised results for business decisions.

■ Skills You Will Gain

- Apply K-Means clustering with elbow method and silhouette score
- Use DBSCAN for density-based clustering and outlier detection
- Reduce dimensions with PCA — select components by explained variance
- Visualise high-dimensional data with t-SNE and UMAP
- Detect anomalies with Isolation Forest and One-Class SVM
- Evaluate cluster quality with silhouette score and inertia

■ Unsupervised learning powers Netflix recommendations, customer segmentation at Zomato/Swiggy, fraud detection, and genomics research. No labels needed.

SECTION 1

K-Means Clustering

1	Choose K Decide number of clusters. Use elbow method or silhouette score to find optimal K
2	Initialise Place K centroids using K-Means++ (smarter than random, avoids bad initialisations)
3	Assign Assign each data point to the nearest centroid by Euclidean distance
4	Update Move each centroid to the mean position of all points assigned to it
5	Repeat Iterate assign + update until centroids stop moving (convergence)
6	Evaluate Check inertia (lower=better) and silhouette score (closer to 1=better)

```
from sklearn.cluster import KMeans from sklearn.metrics import silhouette_score import
matplotlib.pyplot as plt # Find optimal K using elbow + silhouette inertias, silhouettes = [], []
for k in range(2, 11): km = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=10)
km.fit(X_scaled) inertias.append(km.inertia_) silhouettes.append(silhouette_score(X_scaled,
km.labels_)) # Pick K at elbow / max silhouette best_k = silhouettes.index(max(silhouettes)) + 2
km_final = KMeans(n_clusters=best_k, random_state=42, n_init=10) df['Cluster'] =
km_final.fit_predict(X_scaled) # Cluster profiles print(df.groupby('Cluster').mean().round(2))
```

SECTION 2

DBSCAN — Density-Based Clustering

DBSCAN Advantages over K-Means

K-Means: requires K upfront, assumes spherical clusters, outliers distort centroids DBSCAN: finds K automatically, handles arbitrary shapes, labels outliers as -1 (noise) Parameters: eps = neighbourhood radius, min_samples = density threshold Core point: has $\geq$ min_samples neighbours within radius eps Noise point: not a core point and not reachable from any core point — labelled -1

```
from sklearn.cluster import DBSCAN db = DBSCAN( eps=0.5, # neighbourhood radius – tune with
k-distance graph min_samples=5, # density threshold metric='euclidean' ) df['Cluster'] =
db.fit_predict(X_scaled) n_clusters = len(set(df.Cluster)) - (1 if -1 in df.Cluster.values else 0)
n_noise = (df.Cluster == -1).sum() print(f'Clusters: {n_clusters} | Noise points: {n_noise}')
```

SECTION 3

PCA — Dimensionality Reduction

```
from sklearn.decomposition import PCA import numpy as np # Find minimum components for 95% variance
pca_full = PCA() pca_full.fit(X_scaled) cumvar = np.cumsum(pca_full.explained_variance_ratio_)
n_comp = np.argmax(cumvar >= 0.95) + 1 print(f'{n_comp} components explain 95% of variance') #
Reduce dimensions pca = PCA(n_components=n_comp) X_pca = pca.fit_transform(X_scaled) # 2D
visualisation with t-SNE from sklearn.manifold import TSNE tsne = TSNE(n_components=2,
```

```
perplexity=30, random_state=42) X_2d = tsne.fit_transform(X_scaled[:3000]) # use sample for speed
import seaborn as sns sns.scatterplot(x=X_2d[:,0], y=X_2d[:,1], hue=labels, palette='tab10')
```

SECTION 4

Anomaly Detection

	How It Works	Best For
	Isolates anomalies with random splits — they need fewer splits	General tabular anomaly detection
	Learns a boundary around 'normal' data in kernel space	High-dimensional data, text
	Compares local density of each point vs its neighbours	Clusters with varying density
	$ x - \text{mean} / \text{std} > 3 = \text{anomaly}$	Normally distributed features
	$x < Q1 - 1.5 * IQR$ or $x > Q3 + 1.5 * IQR$	Any distribution, robust

```
from sklearn.ensemble import IsolationForest iso = IsolationForest( n_estimators=200,
contamination=0.05, # expected fraction of outliers random_state=42 ) df['anomaly'] =
iso.fit_predict(X_scaled) # -1 = anomaly / outlier, +1 = normal print(f'Anomalies detected:
{(df.anomaly == -1).sum()}') anomalies = df[df.anomaly == -1]
```

■ PRACTICE Q & A

Q1. What is the elbow method for choosing K in K-Means?

A: Plot inertia (within-cluster sum of squares) vs number of clusters K. The 'elbow' point where inertia stops decreasing sharply indicates the optimal K.

Q2. What does DBSCAN label as cluster -1?

A: Noise points — data points that don't belong to any cluster because they are not within eps distance of min_samples other points. Effectively outlier detection.

Q3. What does the first PCA component represent?

A: The linear combination of original features that captures the maximum variance in the data — the direction of greatest spread.

Q4. What is silhouette score and what values are good?

A: Measures how similar each point is to its own cluster vs nearest other cluster. Range [-1, 1]. Score near 1 = well-separated clusters, near 0 = overlapping, negative = wrong assignment.

Q5. When would you use t-SNE instead of PCA for visualisation?

A: t-SNE preserves local neighbourhood structure and reveals non-linear clusters — use for 2D/3D visualisation. PCA is linear and faster — use for preprocessing before modelling.

■ Unsupervised Learning Cheat Sheet	
<code>KMeans(n_clusters=k)</code>	K-Means clustering
<code>DBSCAN(eps=0.5, min_samples=5)</code>	Density clustering
<code>silhouette_score(X, labels)</code>	Cluster quality [-1, 1]
<code>km.inertia_</code>	Within-cluster sum of squares
<code>km.cluster_centers_</code>	Centroid coordinates
<code>PCA(n_components=2)</code>	Linear dimensionality reduction
<code>TSNE(n_components=2)</code>	Non-linear 2D visualisation
<code>IsolationForest(contamination)</code>	Anomaly detection
<code>LocalOutlierFactor(n_neighbors)</code>	LOF anomaly detection

<code>pca.explained_variance_ratio_</code>	Variance per component
<code>np.cumsum(evr) >= 0.95</code>	Find components for 95% variance
<code>AgglomerativeClustering()</code>	Hierarchical clustering